

WorkoutLang

CSE 341 — Concepts of Programming Languages | Gebze Technical University

D3: Example Programs & Test Report (Part 2)

Student: 210104004053 | Due: 22 May 2026 | Implementation: C

This report documents end-to-end execution of all three WorkoutLang example programs under the complete interpreter (lexer → parser → type checker → interpreter), and presents three targeted error programs demonstrating type-checker and runtime error detection. All programs were run with `./workoutlang <file.wl>`. Output shown is the exact stdout/stderr produced by the binary built with `gcc -Wall -Wextra -std=c11`.

Part A — Valid Programs — Full Execution

Program 1 — valid1.wl

Constructs: workout, day, set, routine, int, load, if/else, repeat N weeks

Source Code

```
// Test Program 1: Upper body workout with a routine
workout UpperBody {
  day Monday {
    set benchpress reps 4 x 10 rest 90s
    set overhead_press reps 3 x 8 rest 60s
    set tricep_dip reps 3 x 12 rest 45s
  }
  day Wednesday {
    set pullup reps 4 x 8 rest 90s
    set row reps 3 x 10 rest 60s
  }
}
routine myWeek {
  int totalSets = 5
  load UpperBody
  if totalSets > 3 {
    print "Heavy week"
  } else {
    print "Light week"
  }
  repeat 4 weeks {
    load UpperBody
  }
}
```

Interpreter Output

```
✓ Type checking passed | Execution successful
[UpperBody - Monday]
  set benchpress: 4 x 10, rest 90s
  set overhead_press: 3 x 8, rest 1min
  set tricep_dip: 3 x 12, rest 45s
[UpperBody - Wednesday]
  set pullup: 4 x 8, rest 90s
  set row: 3 x 10, rest 1min
Heavy week
[UpperBody - Monday]
  set benchpress: 4 x 10, rest 90s
... (UpperBody repeated 4 more times - 5 total: 1 load + 4 repeat weeks)
```

Discussion

This program demonstrates the two primary domain-specific constructs: the workout/day/set declaration hierarchy and the load statement. When load UpperBody executes, the interpreter walks every DayDecl node inside the workout and prints one header line per day followed by each set entry — the runtime realisation of the S-Set semantic rule in D1 §4.4 (where $\sigma.\log$ is realised as stdout). The if totalSets > 3 branch correctly evaluates to true and prints Heavy week, demonstrating that the type checker (§4.5) has already verified the condition is bool before execution. The repeat 4 weeks loop executes its body exactly four times with no accessible counter variable, matching the deliberate restriction in D1 §4.8: the count is read directly from the AST integer literal, evaluated once. Duration literals are printed in the most concise unit — 60s appears as 1min — confirming durations are stored internally as seconds and converted on output.

Program 2 — valid2.wl

Constructs: func, return, while, block-scope var_decl, func_call, if

Source Code

```
// Test Program 2: Functions, while loop, expressions
func totalVolume(s: int, r: int) : int {
  return s * r
}
workout LegDay {
  day Friday {
    set squat reps 5 x 5 rest 3min
    set legpress reps 4 x 12 rest 90s
    set lunge reps 3 x 10 rest 60s
  }
}
routine strengthPlan {
  int week = 1
  while week <= 12 {
    load LegDay
    int vol = totalVolume(5, 5)
    if vol > 20 {
      print "High volume"
    }
    week = week + 1
  }
}
```

Interpreter Output

```
✓ Type checking passed | Execution successful
[LegDay - Friday]
  set squat: 5 x 5, rest 3min
  set legpress: 4 x 12, rest 90s
  set lunge: 3 x 10, rest 1min
High volume
[LegDay - Friday]
... (pattern repeats - 12 iterations total, while week <= 12)
```

Discussion

This program exercises the function call pipeline end-to-end: `totalVolume(5, 5)` is called with two int arguments passed by value (D1 §4.6, Sebesta §9.2). The function opens a new `SCOPE_FUNCTION` frame, binds `s=5` and `r=5` as mutable local copies, evaluates `s*r=25`, fires a `ReturnSignal` back to the call site, and the frame is immediately destroyed — demonstrating stack-dynamic lifetime of parameters (§4.6). The while loop re-evaluates `week <= 12` on every iteration against the updated environment, matching the `W-True` rule in D1 §4.4. The variable `vol` is declared inside the loop body and is stack-dynamic (destroyed on each `scope_exit`), so re-declaring it on the next iteration is legal. Because `vol` is always `25 > 20`, `High volume` prints on all 12 iterations; the while exits cleanly when `week` reaches 13.

Program 3 — valid3.wl

Constructs: func returning bool, repeat, load (multi-day), && operator, unary !, if

Source Code

```
// Test Program 3: All types, nested if, bool expressions
func isHeavy(s: int, r: int) : bool {
    return s * r > 30
}
workout FullBody {
    day Monday {
        set deadlift reps 3 x 5 rest 3min
        set benchpress reps 4 x 8 rest 2min
    }
    day Thursday {
        set squat reps 4 x 6 rest 2min
        set overhead_press reps 3 x 10 rest 90s
    }
}
routine periodization {
    repeat 8 weeks {
        load FullBody
        bool heavy = isHeavy(4, 8)
        if heavy && true {
            print "Deload recommended"
        }
        if !heavy {
            print "Add weight next session"
        }
    }
}
```

Interpreter Output

```
✓ Type checking passed | Execution successful
[FullBody - Monday]
  set deadlift: 3 x 5, rest 3min
  set benchpress: 4 x 8, rest 2min
[FullBody - Thursday]
  set squat: 4 x 6, rest 2min
  set overhead_press: 3 x 10, rest 90s
Deload recommended
[FullBody - Monday]
... (pattern repeats - 8 iterations total, repeat 8 weeks)
```

Discussion

This program exercises the full type system with a function returning bool. `isHeavy(4, 8)` evaluates $4*8=32 > 30 = \text{true}$; the return value is bound to the block-local variable `heavy`. The expression `heavy && true` demonstrates short-circuit evaluation (D1 §4.7): the interpreter evaluates the left operand first and only evaluates the right if needed — both must be bool as verified by the type checker. Because `heavy` is true, the first branch prints `Deload recommended`; the second branch (`!heavy`) is false so `Add weight next session` is never printed, confirming only one branch executes per iteration. The `load FullBody` statement executes both day blocks in declaration order on each of the 8 repeat iterations, producing 16 day-header lines across the full output.

Part B — Error Programs — Type Checker and Runtime

The three programs below each contain a deliberate error. E1 and E3 are caught by the type checker (before execution begins); E2 reaches the interpreter and is caught at runtime. All programs exit with a non-zero status and an informative error message on `stderr` in the format `[line N] <Error type>: <message>`.

E1 — Bool variable initialised with int expression

Type rule: `var_decl` type mismatch (D1 §4.5) — declared type must match initialiser type exactly; no implicit coercion.

Source Code

```
routine badDecl {
  int volume = 5
  bool result = volume
}
```

Error Output (stderr)

```
[line 3] Type error: cannot initialise 'result' (type bool) with expression of
type int
Type checking failed - 1 error(s).
```

The type checker catches this in Pass 2 during `STMT_VAR_DECL` processing: declared type is bool and the int variable `volume` resolves to type int. Because `WorkoutLang` has no implicit coercion (§4.5), the mismatch is a type error regardless of value. The checker handles both literal and variable initialisers the same way — only the declared type matters.

E2 — Division by zero (runtime)

Type rule: Runtime integer division by zero (D1 §4.7) — the / operator checks the divisor at runtime and halts execution.

Source Code

```
func divide(num: int, den: int) : int {  
    return num / den  
}  
routine test {  
    int result = divide(10, 0)  
    print result  
}
```

Error Output (stderr)

```
[line 2] Runtime error: division by zero  
Execution failed - 1 runtime error(s).
```

The type checker passes this program — both arguments are int and the function signature matches. The error is detected at runtime inside the interpreter's `EXPR_BINOP` handler when the divisor evaluates to zero. The interpreter prints the error with the line number of the division expression and exits with a non-zero code.

E3 — if condition is int, not bool

Type rule: if-condition type rule (D1 §4.7) — the condition of an if statement must be exactly bool; no implicit coercion from int.

Source Code

```
routine badIf {  
    int count = 5  
    if count {  
        print "never"  
    }  
}
```

Error Output (stderr)

```
[line 3] Type error: 'if' condition must be bool, got int (§4.7)  
Type checking failed - 1 error(s).
```

This error is caught in Pass 2 during `STMT_IF` processing. The type checker evaluates the condition expression `count`, which resolves to `int` via symbol table lookup, and rejects it because §4.7 requires exactly `bool`. Unlike C, WorkoutLang has no notion of truthy integers — `if count` is always a type error regardless of value.

All programs compiled and executed on the workoutlang binary built from `lexer.c / parser.c / symbol_table.c / type_checker.c / interpreter.c` with `gcc -Wall -Wextra -std=c11`. Zero warnings produced.